Idiosyncratic Volatility Dispersion Trading: A Quantitative Pipeline for Cross-Sectional Alpha

Charles Chou

April 6, 2026

Abstract

This document presents a comprehensive mathematical framework for trading index dispersion using idiosyncratic volatility decomposition. The pipeline optimizes constituent basket composition, computes beta-adjusted weights, aggregates idiosyncratic volatility metrics, fits predictive regression models, and validates robustness via walk-forward analysis. Local minima verification ensures numerical stability of the optimization.

1 Introduction and Subject

1.1 The Core Concept

Index dispersion trading exploits the difference between *implied correlation* (priced by options markets) and *realized correlation* (observed in historical returns). The strategy is:

- **Long Dispersion**: Sell index options (expensive implied vol), buy single-stock options (cheaper implied vol). Profit when realized correlation exceeds implied.
- **Short Dispersion**: Buy index options, sell single-stock options. Profit when correlation breaks down.

1.2 The Idiosyncratic Volatility Insight

Traditional dispersion trading relies on implied vs. realized volatility spreads. The idiosyncratic volatility approach adds a *regime detection* layer:

Definition 1 (Idiosyncratic Volatility)**.** *The component of stock return variance unexplained by market movements:*

$$\sigma^2_{idio,i} = Var(\varepsilon_i) \quad where \quad r_{i,t} = \alpha_i + \beta_i r_{m,t} + \varepsilon_{i,t} \tag{1}$$

The ratio $\frac{\sum_i w_i \sigma_{idio,i}}{\sigma_m}$ measures the *cross-sectional dispersion* relative to market volatility, providing a regime-independent signal.

2 Mathematical Framework

2.1 Return Decomposition

For each constituent i and time t:

$$r_{i,t} = \underbrace{\alpha_i}_{\text{Jensen's alpha}} + \underbrace{\beta_i}_{\text{Market beta}} \cdot r_{m,t} + \underbrace{\varepsilon_{i,t}}_{\text{Idiosyncratic shock}} \tag{2}$$

Estimated via rolling OLS over window τ:

$$\hat{\beta}_{i,t} = \frac{\mathrm{Cov}_\tau(r_i, r_m)}{\mathrm{Var}_\tau(r_m)} \tag{3}$$

$$\hat{\alpha}_{i,t} = \bar{r}_i - \hat{\beta}_{i,t}\bar{r}_m \tag{4}$$

$$\hat{\varepsilon}_{i,t} = r_{i,t} - \hat{\alpha}_{i,t} - \hat{\beta}_{i,t} r_{m,t} \tag{5}$$

2.2 Idiosyncratic Volatility

Annualized idiosyncratic volatility:

$$\sigma_{idio,i,t} = \sqrt{\frac{\sum_{s=t-\tau}^{t} \hat{\varepsilon}_{i,s}^2}{\tau - 2}} \cdot \sqrt{252} \tag{6}$$

2.3 Basket Aggregation

For weights $\mathbf{w} = (w_1, \ldots, w_n)$ where $\sum_i w_i = 1$:

$$\sigma_{basket,realized} = \sum_{i=1}^{n} w_i \sigma_{realized,i} \tag{7}$$

$$\sigma_{basket,implied} = \sum_{i=1}^{n} w_i \sigma_{implied,i} \tag{8}$$

$$\sigma_{basket,idio} = \sum_{i=1}^{n} w_i \sigma_{idio,i} \tag{9}$$

2.4 Key Dispersion Metrics

$$\text{Dispersion Spread}_t = \sigma_{index,implied,t} - \sigma_{basket,implied,t} \tag{10}$$

$$\text{Idiosyncratic Ratio}_t = \frac{\sigma_{basket,idio,t}}{\sigma_{index,realized,t}} \tag{11}$$

2.5 Predictive Regression

$$\underbrace{(\sigma_{m,t+\delta}^{realized} - \sigma_{basket,t+\delta}^{realized})}_{\text{Future Realized Dispersion}} = \alpha + \beta_1 \cdot \underbrace{\text{Dispersion Spread}_t}_{\text{Current Implied}} + \beta_2 \cdot \underbrace{\text{Idio Ratio}_t}_{\text{Regime Signal}} + \beta_3 \cdot (\text{Dispersion}_t \times \text{Ratio}_t) + \varepsilon_t \tag{12}$$

Expected coefficients: $\beta_1 > 0$ (implied predicts realized), $\beta_2 < 0$ (high idio predicts mean reversion).

3 The Pipeline

3.1 Overview

The system implements six core functions executed sequentially:

1. ComputePortfolioWeights — Constituent weight calculation
2. AggregateIdiosyncraticMetrics — Basket metric aggregation
3. FitDispersionRegression — Predictive signal modeling

4. ValidateSignalRobustness — Walk-forward testing
5. OptimizeBasketComposition — Local minima verified optimization
6. RunFullDispersionOptimization — Pipeline orchestration

3.2 Algorithm 1: Weight Calculation

Algorithm 1 ComputePortfolioWeights

Require: DataFrame D with multi-index $(ticker, field, tenor)$, tenor τ, index tickers $\mathcal{I}$, basket tickers $\mathcal{B}$, method $m \in \{\text{MarketCap}, \text{Index}, \text{Beta}\}$

Ensure: Weight vector $\mathbf{w}$ with $\sum_i w_i = 1$

```
Extract market caps: mc_i ← D_(i,market_cap,τ) for i ∈ B
if m = MarketCap then
    w_i ← mc_i / Σ_j mc_j
else if m = Index then
    Compute betas: β_i ← Cov_τ(r_i, r_index) / Var_τ(r_index) for i ∈ B
    w_i^scaled ← mc_i · β_i
    w_i ← w_i^scaled / Σ_j w_j^scaled
else if m = Beta then
    Compute betas: β_i as above
    w_i^raw ← mc_i / β_i                                  ▷ Inverse beta weighting
    w_i ← w_i^raw / Σ_j w_j^raw
end if
return {i : w_i | i ∈ B, w_i > 0} normalized
```

Mathematical Formulations:

$$\text{MarketCap:} \quad w_i = \frac{\text{MC}_i}{\sum_j \text{MC}_j} \tag{13}$$

$$\text{Index:} \quad w_i = \frac{\text{MC}_i \cdot \beta_i}{\sum_j \text{MC}_j \cdot \beta_j} \tag{14}$$

$$\text{Beta:} \quad w_i = \frac{\text{MC}_i / \beta_i}{\sum_j \text{MC}_j / \beta_j} \tag{15}$$

3.3 Algorithm 2: Metric Aggregation

Algorithm 2 AggregateIdiosyncraticMetrics

Require: DataFrame D, weights $\mathbf{w}$, tenor τ, index ticker m, lookback L
Ensure: Metrics DataFrame M with time series

```
for each stock i ∈ keys(w) do
    for each time t ∈ [L, T] do
        Extract returns window: r_i ← D_(i,returns,τ)[t − L : t]
        Extract index window: r_m ← D_(m,returns,τ)[t − L : t]
        X ← [1, r_m]                                          ▷ Design matrix
        (α̂_i, β̂_i) ← OLS(r_i, X)
        ε̂ ← r_i − α̂_i − β̂_i r_m
        σ_idio,i,t ← std(ε̂) · √252
    end for
end for
for each time t do
    σ_basket,realized,t ← Σ_i w_i · σ_realized,i,t
    σ_basket,implied,t ← Σ_i w_i · σ_implied,i,t
    σ_basket,idio,t ← Σ_i w_i · σ_idio,i,t
    Spread_t ← σ_m,implied,t − σ_basket,implied,t
    Ratio_t ← σ_basket,idio,t / σ_m,realized,t
end for
return DataFrame M with columns {basket_realized, basket_implied, basket_idio, index_realized, index_impl
```

3.4 Algorithm 3: Signal Regression

Algorithm 3 FitDispersionRegression

Require: Metrics M, signal column s, regime column r, forward days δ, lag_dependent flag, interaction flag
Ensure: RegressionResult with coefficients and diagnostics

```
y ← Future Realized Dispersion at t + δ                 ▷ y_t = σ^real_m,t+δ − σ^real_basket,t+δ
X_1 ← M[s]                                              ▷ Signal: dispersion spread
X_2 ← M[r]                                              ▷ Regime: idiosyncratic ratio
if interaction then
    X_3 ← X_1 ⊙ X_2                                     ▷ Element-wise product
    X ← [1, X_1, X_2, X_3]
else
    X ← [1, X_1, X_2]
end if
θ̂ ← (X^T X)^−1 X^T y                                    ▷ OLS estimation
ŷ ← X θ̂
R^2 ← 1 − Σ(y − ŷ)^2 / Σ(y − ȳ)^2
if lag_dependent then
    PredictivePower ← Cor(ŷ_train, y_test)^2
end if
return θ̂, R^2, residuals, ŷ
```

Regression Model:

$$y_{t+\delta} = \theta_0 + \theta_1 x_{1,t} + \theta_2 x_{2,t} + \theta_3 (x_{1,t} \cdot x_{2,t}) + \varepsilon_t \tag{16}$$

3.5 Algorithm 4: Robustness Validation

Algorithm 4 ValidateSignalRobustness

Require: Metrics M, weights $\mathbf{w}$, regression coefficients $\hat{\theta}$, train fraction γ, splits K, rebalance freq f, cost c

Ensure: RobustnessResult with stability metrics

$N \leftarrow \text{length}(M)$, $n_{split} \leftarrow \lfloor N/K \rfloor$
Threshold grid $\mathcal{T} \leftarrow \{50, 100, 150, 200, 300\}$ bps
for $k = 1$ to $K - 1$ **do** ▷ Walk-forward splits
  $train \leftarrow M[0 : k \cdot n_{split}]$
  $test \leftarrow M[k \cdot n_{split} : (k+1) \cdot n_{split}]$
  for $\tau \in \mathcal{T}$ **do**
    $\text{P\&L} \leftarrow \text{SimulateTrading}(test, \hat{\theta}, \tau, f, c)$
    $\text{Sharpe}_k(\tau) \leftarrow \frac{\text{mean(P\&L)}}{\text{std(P\&L)}} \cdot \sqrt{252}$
  end for
  $\tau_k^* \leftarrow \arg\max_\tau \text{Sharpe}_k(\tau)$
  Store $(\tau_k^*, \text{Sharpe}_k(\tau_k^*))$
end for
$\text{StabilityScore} \leftarrow 1 - \frac{\text{std}(\{\text{Sharpe}_k\})}{\text{mean}(\{\text{Sharpe}_k\}) + \epsilon}$
$\text{OptimalThreshold} \leftarrow \text{median}(\{\tau_k^*\})$
return StabilityScore, OptimalThreshold, WalkForwardResults

Trading Simulation Subroutine:

$$\text{Position}_t = \begin{cases} +1 & \text{if } \hat{y}_{t+\delta} > \tau \\ -1 & \text{if } \hat{y}_{t+\delta} < -\tau \\ 0 & \text{otherwise} \end{cases} \tag{17}$$

$$\text{P\&L}_t = \text{Position}_{t-1} \cdot (\sigma_{m,t}^{real} - \sigma_{basket,t}^{real}) - |\Delta\text{Position}_t| \cdot c \tag{18}$$

3.6 Algorithm 5: Basket Optimization with Local Minima Verification

Algorithm 5 OptimizeBasketComposition (Gradient Descent with Verification)

Require: DataFrame D, tenor τ, index m, candidates $\mathcal{C}$, target size n, objective f, max iterations T, tolerance ϵ, neighbors K

Ensure: Optimal basket $\mathcal{B}^*$, weights $\mathbf{w}^*$, local minimum flag

Initialization: $\mathcal{B}^{(0)} \leftarrow$ Top-n by preliminary score
$t \leftarrow 0$, converged $\leftarrow$ False
while $t < T$ and not converged **do**
    $S_{current} \leftarrow f(\mathcal{B}^{(t)})$ ▷ Evaluate objective
    Find best swap:
    for $i \in \mathcal{B}^{(t)}$ **do** ▷ Remove candidate
        $\mathcal{B}^- \leftarrow \mathcal{B}^{(t)} \setminus \{i\}$
        $S_i^- \leftarrow f(\mathcal{B}^-)$
        for $j \in \mathcal{C} \setminus \mathcal{B}^{(t)}$ **do** ▷ Add candidate
            $\mathcal{B}^+ \leftarrow \mathcal{B}^- \cup \{j\}$
            $S_{i,j}^+ \leftarrow f(\mathcal{B}^+)$
            Improvement$(i, j) \leftarrow S_{i,j}^+ - S_{current}$
        end for
    end for
    $(i^*, j^*) \leftarrow \arg\max_{i,j}$ Improvement(i, j)
    if Improvement$(i^*, j^*) > \epsilon$ **then**
        $\mathcal{B}^{(t+1)} \leftarrow (\mathcal{B}^{(t)} \setminus \{i^*\}) \cup \{j^*\}$
        $t \leftarrow t + 1$
    else
        converged $\leftarrow$ True
    end if
end while
$\mathcal{B}^* \leftarrow \mathcal{B}^{(t)}$
Local Minima Verification:
neighbors $\leftarrow$ []
for $i \in \mathcal{B}^*$ and $k = 1$ to K **do**
    $j \leftarrow k$-th best alternative from $\mathcal{C} \setminus \mathcal{B}^*$
    $\mathcal{N}_k \leftarrow (\mathcal{B}^* \setminus \{i\}) \cup \{j\}$
    neighbors.append$(f(\mathcal{N}_k))$
end for
if $f(\mathcal{B}^*) >$ max(neighbors) **then**
    is_local_min $\leftarrow$ True
else
    is_local_min $\leftarrow$ False ▷ Could restart from best neighbor if desired
end if
$\mathbf{w}^* \leftarrow$ ComputePortfolioWeights$(D, \tau, [m], \mathcal{B}^*)$
return $\mathcal{B}^*, \mathbf{w}^*, f(\mathcal{B}^*)$, is_local_min

Objective Functions:

$$f_{\text{dispersion}}(\mathcal{B}) = \mathbb{E}[\text{IdioRatio}(\mathcal{B})] - \lambda \cdot \text{TrackingError}(\mathcal{B}) \tag{19}$$

$$f_{\text{tracking}}(\mathcal{B}) = -\text{TrackingError}(\mathcal{B}) \tag{20}$$

$$f_{\text{hybrid}}(\mathcal{B}) = w_1 \cdot \text{Sharpe}(\mathcal{B}) - w_2 \cdot \text{Turnover} - w_3 \cdot \text{TrackingError} \tag{21}$$

Local Minimum Criterion:

$$\mathcal{B}^* \text{ is local minimum} \iff f(\mathcal{B}^*) > f(\mathcal{N}_k) \quad \forall k \in \{1, \ldots, K\} \tag{22}$$

$$\text{Margin} = \min_k [f(\mathcal{B}^*) - f(\mathcal{N}_k)] > 2 \cdot \sigma_{error} \tag{23}$$

3.7 Algorithm 6: Pipeline Orchestration

Algorithm 6 RunFullDispersionOptimization

Require: DataFrame D, tenor τ, index m, candidates $\mathcal{C}$, config $\mathcal{P}$
Ensure: CompleteResults with basket, metrics, model, validation, recommendations

```
Step 1: Basket Optimization
(B, w, obj, verified) ← OptimizeBasketComposition(D, τ, m, C, P)
Step 2: Weight Refinement
w ← ComputePortfolioWeights(D, τ, [m], B, P.weighting_method)
Step 3: Metric Aggregation
M ← AggregateIdiosyncraticMetrics(D, w, τ, m)
Step 4: Signal Regression
R ← FitDispersionRegression(M, "dispersion_spread", "idiosyncratic_ratio")
Step 5: Robustness Testing
V ← ValidateSignalRobustness(M, w, R)
Step 6: Compile Recommendations
threshold ← V.optimal_threshold
sharpe ← V.mean_sharpe
return { basket: (B, w, verified), metrics: M, regression: R, validation: V, recommendations: (threshold, sharpe) }
```

4 Key Equations Summary

Table 1: Core Mathematical Equations

Concept	**Equation**
Return Decomposition	$r_i = \alpha_i + \beta_i r_m + \varepsilon_i$
Idiosyncratic Volatility	$\sigma_{idio,i} = \sqrt{\text{Var}(\varepsilon_i)}$
Basket Aggregation	$\sigma_{basket} = \sum_i w_i \sigma_i$
Dispersion Spread	$\text{Spread} = \sigma_{m,impl} - \sigma_{basket,impl}$
Idiosyncratic Ratio	$\text{Ratio} = \frac{\sum_i w_i \sigma_{idio,i}}{\sigma_{m,real}}$
Predictive Regression	$y_{t+\delta} = \alpha + \beta_1 x_{1,t} + \beta_2 x_{2,t} + \varepsilon_t$
Weight (MarketCap)	$w_i = \frac{\text{MC}_i}{\sum_j \text{MC}_j}$
Weight (Beta-Adj)	$w_i \propto \frac{\text{MC}_i}{\beta_i}$
Beta Estimation	$\beta_i = \frac{\text{Cov}(r_i, r_m)}{\text{Var}(r_m)}$
Local Minimum	$f(\mathcal{B}^*) > f(\mathcal{N}_k)\ \forall k$

5 Conclusion

The presented pipeline provides a mathematically rigorous framework for idiosyncratic volatility dispersion trading. Key innovations include:

1. **Beta-adjusted weighting** that accounts for cross-sectional differences in market sensitivity
2. **Idiosyncratic ratio normalization** enabling regime-independent signals
3. **Predictive regression** with interaction terms capturing conditional dynamics
4. **Local minima verified optimization** ensuring robust basket selection
5. **Walk-forward validation** preventing overfitting via purged cross-validation

The mathematical consistency across all algorithms—return decomposition → volatility aggregation → signal regression → trading execution— ensures that each component respects the fundamental variance-covariance relationships governing index behavior.